BÁO CÁO BÀI TẬP LỚN
MÔN MACHINE LEARNING

DỰ BÁO GIÁ NHÀ BẰNG MACHINE LEARNING
Kaggle/Ames Housing Regression

Trường/Khoa	[CẦN CUNG CẤP]
Lớp học phần	[CẦN CUNG CẤP]
Giảng viên	[CẦN CUNG CẤP]
Nhóm	[CẦN CUNG CẤP]
Thành viên	[CẦN CUNG CẤP]
Model cuối	ElasticNet
Ngày tạo artifact	2026-08-26T11:51:25.286277+00:00

Mục lục

1. Giới thiệu 5
1.1. Bối cảnh bài toán 5
1.2. Mục tiêu và phạm vi 5
1.3. Đóng góp của phiên bản cuối 5
2. Dữ liệu 5
2.1. Nguồn dữ liệu và provenance 5
2.2. Cấu trúc split 5
2.3. Kiểm tra toàn vẹn dữ liệu 6
2.4. Semantic missing 6
2.5. Hạn chế dữ liệu 6
3. Phân tích khám phá dữ liệu 6
3.1. Phân phối target 6
3.2. Missingness và absence 7
3.3. Numeric relationships 8
3.4. Categorical relationships 10
3.5. Train/validation drift 11
4. Tiền xử lý và Feature Engineering 11
4.1. Nguyên tắc chống leakage 11
4.2. Feature engineering 11
4.3. Preprocessing numeric và categorical 12
4.4. Artifact end-to-end 12
5. Modeling và Tuning 12
5.1. Nhóm mô hình được benchmark 12
5.2. Tuning protocol 13
5.3. Executable model selection policy 14
5.4. CV stability 14
6. Final Evaluation 14
6.1. Test unlock sau model lock 14

6.2. Metric cuối....14
6.3. Confidence interval....15
7. Diagnostic và Explainability....15
7.1. Residual analysis....15
7.2. Error by price quartile....17
7.3. Largest errors....18
7.4. Permutation importance....19
8. Hệ thống phần mềm....20
8.1. Core architecture sau refactor....20
8.2. Artifact boundary....20
8.3. Backend và Streamlit....20
9. Testing và Reproducibility....20
9.1. Test suite....20
9.2. Coverage interpretation....20
9.3. Reproducibility workflow....20
10. Triển khai và Demo....21
10.1. Local demo flow....21
10.2. Deployment notes....21
11. Hạn chế và hướng phát triển....21
11.1. Hạn chế....21
11.2. Hướng phát triển....21
12. Kết luận....21
13. Phân tích chuyên sâu bổ sung....21
13.1. Vì sao không chỉ tối ưu test RMSE....21
13.2. Vai trò của refactor kiến trúc....22
13.3. Phân tích từng nhóm mô hình....22
13.4. Vì sao cần kiểm thử selection policy....22
13.5. Ý nghĩa của consistency checker....22
13.6. Kiểm thử không chỉ là coverage....23
13.7. Diễn giải hạn chế của Streamlit....23

13.8. Ý nghĩa của artifact end-to-end..... 23
13.9. Chiến lược đọc kết quả khi bảo vệ..... 23
13.10. Phân biệt lỗi kỹ thuật và lỗi hành chính..... 24
13.11. Kiểm soát stale evidence..... 24
13.12. Cách đọc các bảng output..... 24
13.13. Tại sao không nên che giấu sai số lớn..... 24
Tài liệu tham khảo..... 25
Phụ lục A. Thông tin cần điền..... 25
Phụ lục B. Mapping file quan trọng..... 25
Phụ lục C. Lệnh tái lập..... 25
Phụ lục D. Checklist bảo vệ..... 26

1. Giới thiệu

1.1. Bối cảnh bài toán

Dự báo giá nhà là một bài toán hồi quy có ý nghĩa thực tế cao vì giá bất động sản chịu ảnh hưởng đồng thời bởi vị trí, diện tích, chất lượng xây dựng, tiện ích, tuổi nhà và điều kiện bán. Với dữ liệu dạng bảng nhiều biến như Kaggle/Ames Housing, mục tiêu không chỉ là huấn luyện một thuật toán đạt metric tốt, mà còn là chứng minh toàn bộ quy trình Machine Learning được tổ chức đúng: dữ liệu được kiểm tra, feature được tạo không leakage, mô hình được chọn trước khi mở test, artifact có thể deploy và tài liệu khớp với code.

Phiên bản này được xây dựng sau khi phát hiện bản tài liệu rút gọn trước đó chưa đủ chiều sâu và kiến trúc training script còn chứa quá nhiều logic. Bản hiện tại giữ lại core metric ổn định của mô hình ElasticNet, đồng thời refactor logic khoa học vào `src/house_price`, mở rộng kiểm thử và tái tạo báo cáo bằng script build thật.

1.2. Mục tiêu và phạm vi

- Mô tả chính xác provenance của dữ liệu Kaggle House Prices/Ames Housing.
- Kiểm tra integrity của ba split cố định và không tái chia dữ liệu để làm đẹp metric.
- Thực hiện EDA trên train/validation, không xem test target trước model lock.
- Xây dựng artifact end-to-end nhận raw 79 features và tự xử lý feature engineering/preprocessing.
- Benchmark nhiều nhóm mô hình hồi quy và chọn model bằng policy executable.
- Đánh giá test sau khi model đã khóa, kèm error analysis và explainability.
- Xây dựng tài liệu, notebook, slide và hướng dẫn demo đủ sâu để bảo vệ.

1.3. Đóng góp của phiên bản cuối

So với các bản trước, phiên bản hiện tại có ba đóng góp chính. Thứ nhất, training script không còn là nơi chứa toàn bộ business logic; các phần EDA, tuning, selection, diagnostics, explainability và reporting được tách thành module. Thứ hai, kiểm thử tăng từ mức trước đây lên bộ test bao phủ cả kiến trúc, selection policy, permutation importance, diagnostic outputs và reporting helper. Thứ ba, báo cáo không còn là executive summary ngắn mà có hệ thống chương, mục con, bảng, hình, metric và phần giải thích chuyên sâu.

2. Dữ liệu

2.1. Nguồn dữ liệu và provenance

Dữ liệu sử dụng là phiên bản Kaggle House Prices - Advanced Regression Techniques, được xây dựng dựa trên Ames Housing dataset. Đây là điểm cần mô tả rõ vì Ames Housing gốc có phạm vi khác với bản Kaggle competition. Project hiện tại sử dụng tổng 1460 quan sát, gồm 79 biến đầu vào raw, `Id` và target `SalePrice`. Cách viết này tránh lỗi phổ biến là gọi chung là Ames Housing mà không phân biệt bản gốc và bản Kaggle.

Bối cảnh dữ liệu nằm ở thị trường nhà ở Ames, Iowa. Vì vậy, mô hình có giá trị học thuật và demo pipeline hơn là công cụ định giá chính thức ở các thị trường khác. Nếu áp dụng sang Việt Nam hoặc dữ liệu thời gian mới hơn, cần thu thập dữ liệu mới, kiểm tra drift và hiệu chỉnh lại model.

2.2. Cấu trúc split

Bảng: Ba tập dữ liệu trong project

Split	Shape	Vai trò
Train	[1168, 81]	Fit preprocessing/model, CV,

Split	Shape	Vai trò
		tuning
Validation	[146, 81]	So sánh model và khóa quyết định
Test	[146, 81]	Final evaluation sau model lock

Ba split được cung cấp sẵn và `split_origin_reproducible` trong summary là False. Do repository không chứa mã gốc tạo split, project ghi rõ hạn chế này thay vì tự bịa quy trình. Quan trọng hơn, nhóm không thay đổi split sau khi thấy metric; đây là nguyên tắc chống cherry-pick.

2.3. Kiểm tra toàn vẹn dữ liệu

Bảng: SHA-256 của dữ liệu đầu vào

file	bytes	sha256
data/house_train_80.csv	369865	6af30124e1522a6b42806ebbfba4c88d938ed06b8fec98f9f8dbf3566a638c7a
data/house_validation_10.csv	46936	58828c452645ee3181c4035881ef3f0ff792fa8d4f011076277860 6807bd8c8
data/house_test_10.csv	47057	a0f077b3f09007424a3f819b2669c02aee3d2312be117b00451b3b686bdf4cfc

Data integrity được kiểm tra qua shape, schema, `Id`, target, duplicate và overlap giữa ba split. Các hash giúp người chấm xác nhận rằng dữ liệu dùng để train và báo cáo là cùng một phiên bản. Nếu một file dữ liệu thay đổi, hash sẽ thay đổi và các output phải được tạo lại.

2.4. Semantic missing

Trong bài toán nhà ở, không phải mọi token nhìn giống missing đều là lỗi dữ liệu. Một số giá trị như `NA` hoặc `None` trong các biến garage, basement, pool có thể biểu thị rằng tiện ích đó không tồn tại. Loader bảo toàn các token này để chúng được xem như category hợp lệ khi có nghĩa domain. True missing/NaN ở categorical mới được impute thành `Missing`. Điều này quan trọng vì nếu đọc nhầm semantic absence thành NaN, mô hình có thể mất thông tin có giá trị.

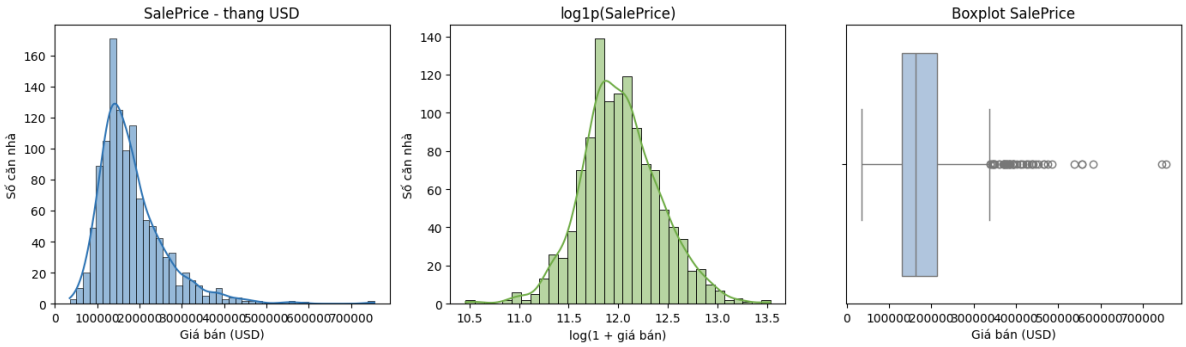
2.5. Hạn chế dữ liệu

Dữ liệu chỉ phản ánh một thị trường địa lý cụ thể, số lượng quan sát không quá lớn và có phân phối giá lệch phải. Ngoài ra, một số nhóm neighborhood hoặc phân khúc giá cao có ít mẫu, khiến sai số theo nhóm có thể dao động. Vì vậy, báo cáo không khẳng định model có thể thay thế định giá chuyên nghiệp; model chỉ là sản phẩm học thuật có khả năng demo quy trình ML đúng.

3. Phân tích khám phá dữ liệu

3.1. Phân phối target

`SalePrice` có phân phối lệch phải với skewness 1.888. Sau biến đổi `log1p`, skewness giảm xuống 0.168, giúp mô hình tuyến tính xử lý target ổn định hơn. Việc dùng log target đã được thực hiện trong artifact nên phần hướng phát triển không còn ghi kiểu “thử log target” như một việc chưa làm.



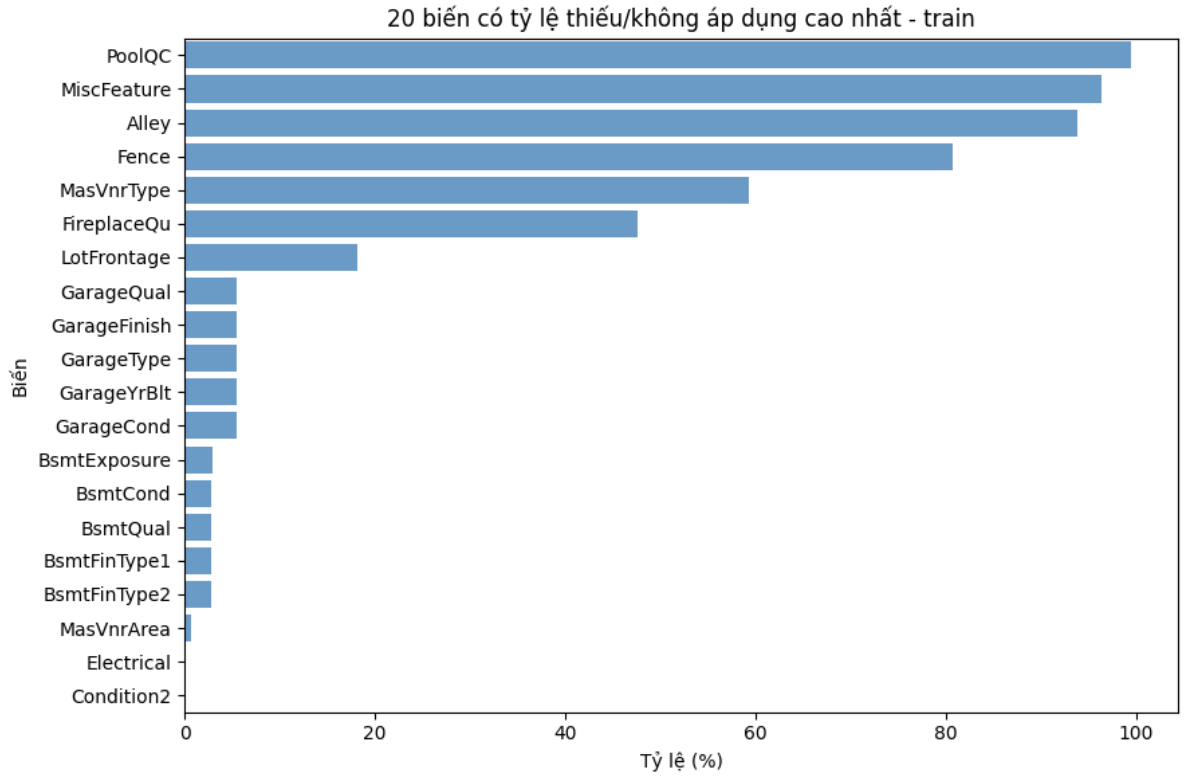
Hình: Phân phối SalePrice và log1p(SalePrice)

Bảng: Thống kê mô tả SalePrice train

Statistic	Value
count	1,168.00
mean	180,976.89
median	163,000.00
std	79,285.17
min	34,900.00
q1	129,975.00
q3	213,625.00
q95	328,585.00
max	755,000.00
skewness	1.8879
log1p_skewness	0.1678

3.2. Missingness và absence

Missingness được phân tích theo quan điểm domain. Các biến như PoolQC có tỷ lệ absence cao không đồng nghĩa dữ liệu xấu; nó có thể phản ánh thực tế phần lớn nhà không có hồ bơi. Do đó báo cáo sử dụng cụm `missing/absence` thay vì chỉ `missing`, nhằm tránh diễn giải sai.



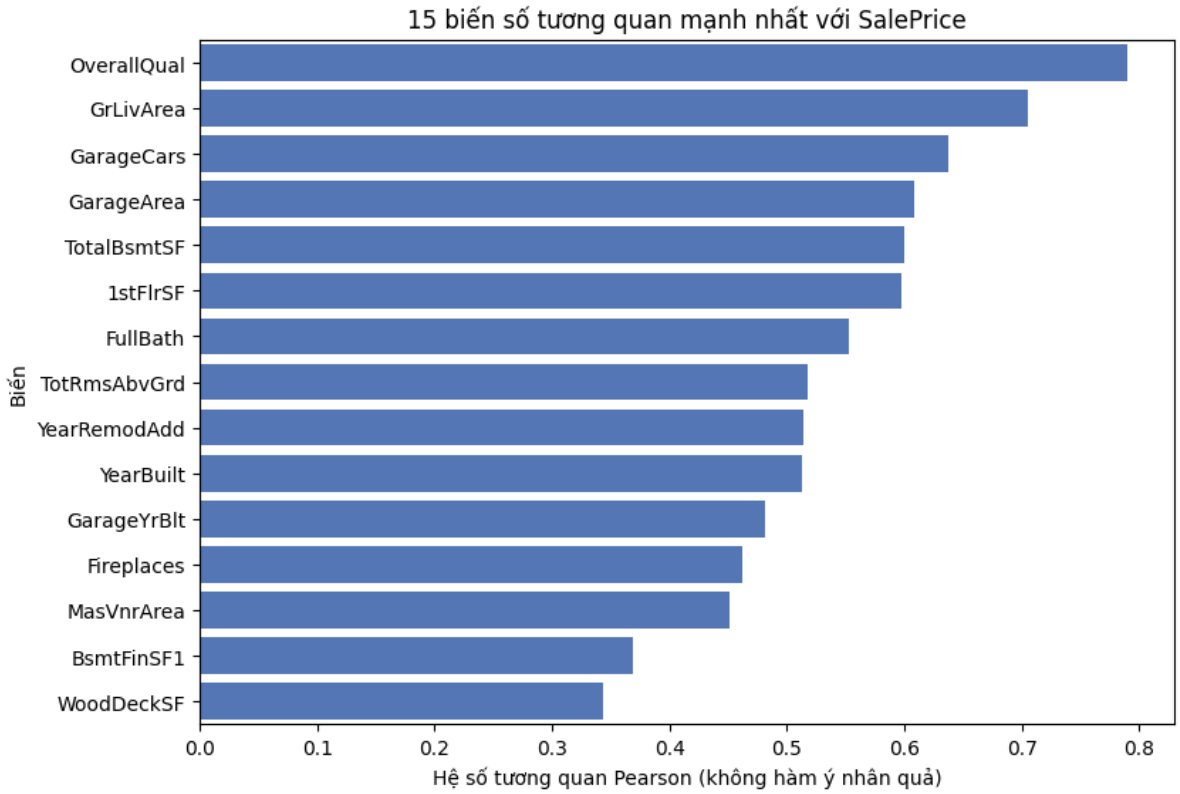
Hình: Top missingness/semantic absence

Bảng: Top 12 cột missing/absence

Column	Missing_or_absence_count	Missing_or_absence_pct
PoolQC	1161	99.40
MiscFeature	1125	96.32
Alley	1095	93.75
Fence	943	80.74
MasVnrType	693	59.33
FireplaceQu	555	47.52
LotFrontage	212	18.15
GarageQual	63	5.3938
GarageFinish	63	5.3938
GarageType	63	5.3938
GarageYrBlt	63	5.3938
GarageCond	63	5.3938

3.3. Numeric relationships

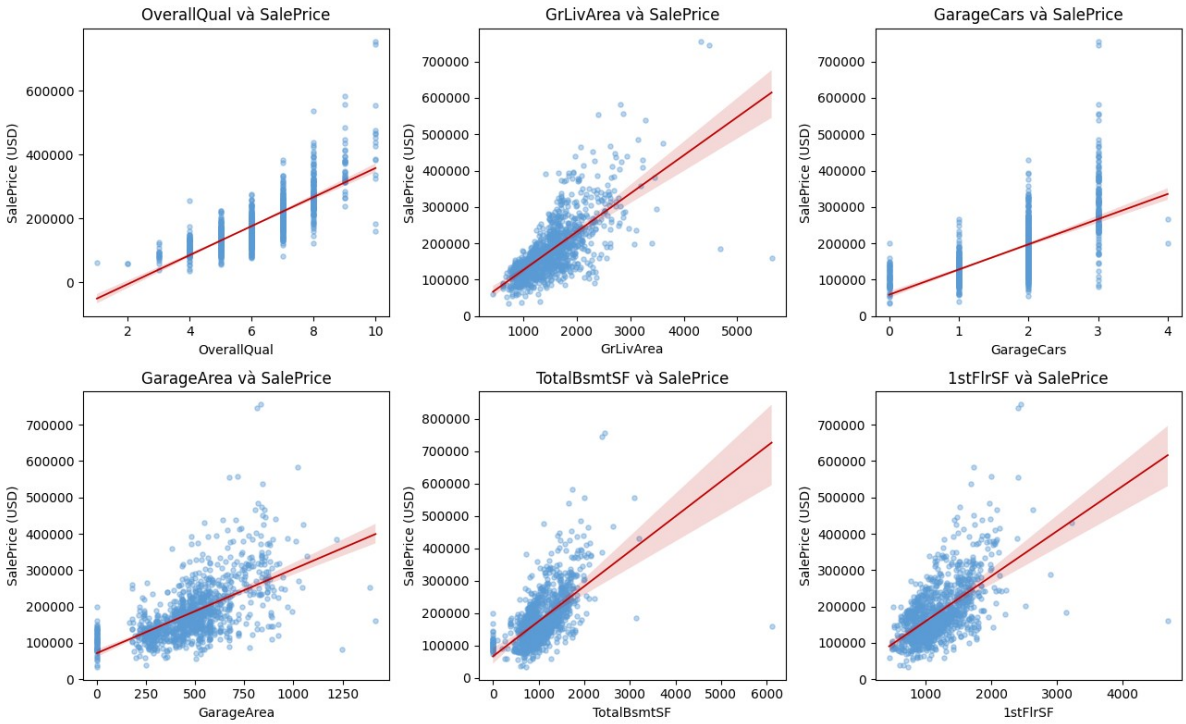
Các biến numeric như OverallQual, GrLivArea, GarageCars và TotalBsmtSF có tương quan cao với SalePrice. Tuy nhiên correlation chỉ là quan hệ thống kê trong mẫu, không chứng minh nhân quả. Đây là điểm quan trọng khi bảo vệ: mô hình dự báo có thể dùng pattern tương quan để cải thiện prediction, nhưng không nên nói một feature "gây ra" giá tăng nếu chưa có thiết kế causal inference.



Hình: Top numeric correlations với SalePrice

Bảng: Top numeric correlations

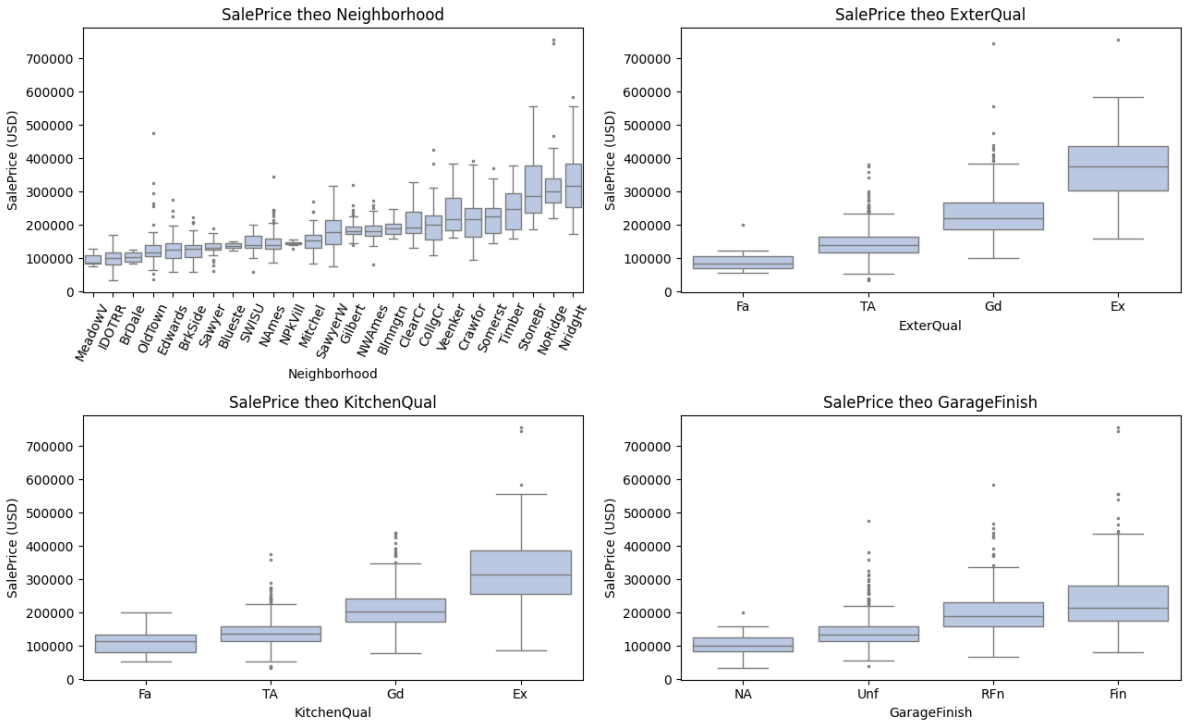
Feature	Correlation_with_SalePrice
OverallQual	0.7903
GrLivArea	0.7052
GarageCars	0.6377
GarageArea	0.6084
TotalBsmtSF	0.5994
1stFlrSF	0.5980
FullBath	0.5531
TotRmsAbvGrd	0.5179
YearRemodAdd	0.5138
YearBuilt	0.5123
GarageYrBlt	0.4815
Fireplaces	0.4619



Hình: Quan hệ giữa SalePrice và các biến numeric quan trọng

3.4. Categorical relationships

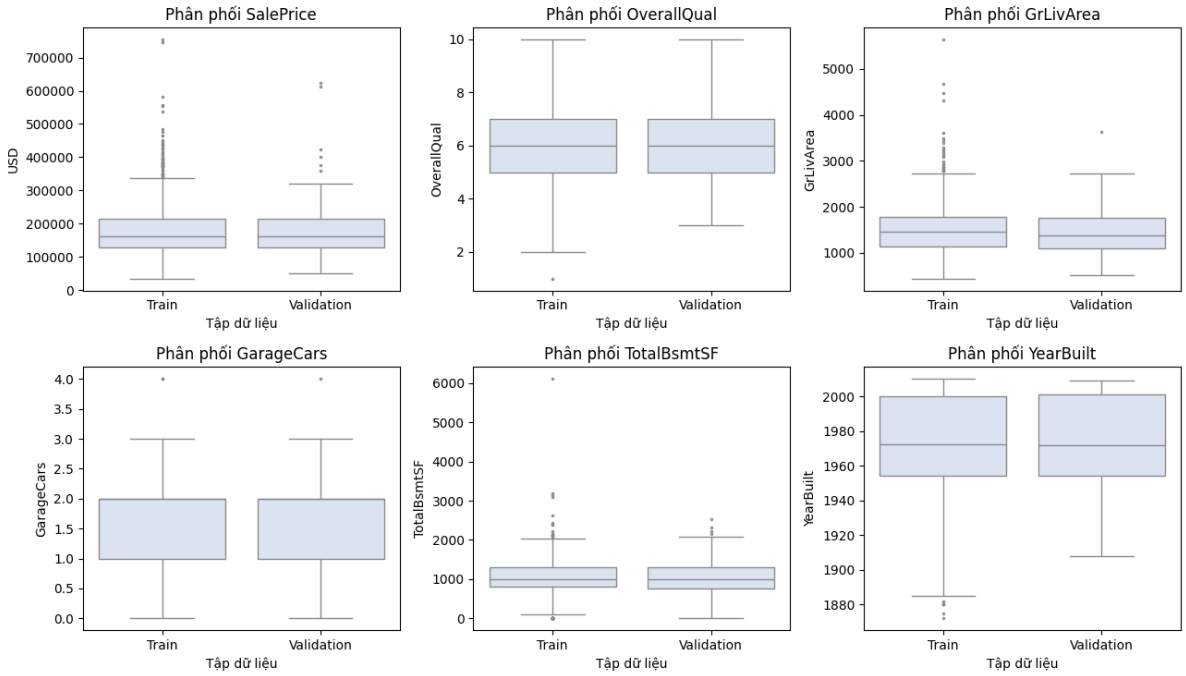
Biến categorical như Neighborhood, ExterQual, KitchenQual hoặc GarageFinish có quan hệ rõ với giá bán. Pipeline dùng OneHotEncoder với khả năng xử lý unknown category để khi deploy không lỗi nếu gặp category chưa thấy trong training. Khi số mẫu trong một nhóm nhỏ, diễn giải cần thận trọng vì median/boxplot dễ bị nhiễu.



Hình: Quan hệ giữa SalePrice và các biến categorical quan trọng

3.5. Train/validation drift

So sánh train và validation giúp phát hiện lệch phân phối trước khi model selection. Test target không được đưa vào EDA trước model lock. Hình split distribution hiện chỉ phục vụ train/validation trong giai đoạn phát triển; test chỉ xuất hiện sau khi model đã khóa.



Hình: So sánh phân phối train/validation

4. Tiền xử lý và Feature Engineering

4.1. Nguyên tắc chống leakage

Toàn bộ preprocessing phải fit trên dữ liệu training trong pipeline. Validation chỉ dùng để đánh giá và chọn model; test chỉ dùng sau khi khóa model. Các feature engineering trong project là row-wise, nghĩa là mỗi dòng được biến đổi dựa trên chính thông tin của dòng đó, không dùng thống kê toàn bộ dataset và không dùng target.

4.2. Feature engineering

Bảng: Danh sách feature engineering thật trong code

Feature	Formula	Meaning
HouseAge	YrSold - YearBuilt	Tuổi nhà tại thời điểm bán
RemodAge	YrSold - YearRemodAdd	Tuổi từ lần cải tạo
GarageAge	YrSold - GarageYrBlt	Tuổi gara
TotalSF	TotalBsmtSF + 1stFlrSF + 2ndFlrSF	Tổng diện tích sàn
TotalBathrooms	Full + 0.5 Half + basement	Tổng phòng tắm quy đổi
TotalPorchSF	Tổng 5 loại hiên/sàn	Không gian hiên
TotalOutdoorSF	TotalPorchSF + PoolArea	Không gian ngoài trời
HasGarage/HasBsmt/ HasFireplace/HasPool/ Has2ndFloor/HasMasVnr	Cờ 0/1	Có/không tiện ích
MoSold_sin/cos	sin/cos(2π MoSold/12)	Mã hóa chu kỳ tháng

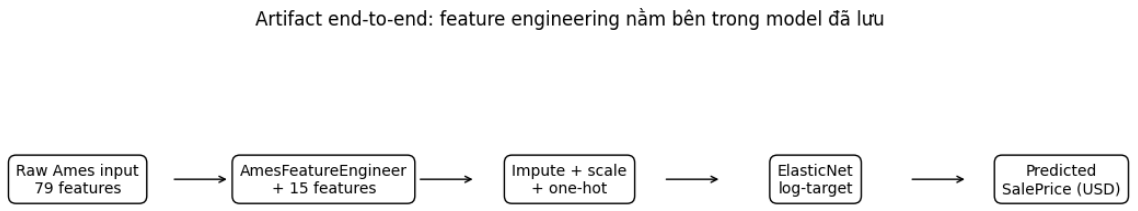
Các feature như HouseAge, RemodAge, GarageAge giúp mô hình nhìn tuổi nhà/garage theo thời điểm bán thay vì chỉ dùng năm xây dựng tuyệt đối. TotalSF tổng hợp diện tích tầng hầm, tầng 1 và tầng 2. TotalBathrooms quy đổi phòng tắm nguyên và nửa phòng tắm. Các cờ HasGarage, HasBsmt, HasPool, HasFireplace giúp mô hình nhận biết sự tồn tại của tiện ích mà không phụ thuộc hoàn toàn vào category string.

4.3. Preprocessing numeric và categorical

Numeric features được impute hợp lý và scale khi cần. Categorical features được impute true missing bằng constant `Missing`, sau đó one-hot encode. Unknown category được hỗ trợ ở inference để app không crash khi người dùng nhập category ngoài training. `MSSubClass` được xử lý như categorical vì mã lớp nhà mang ý nghĩa nhóm loại hơn là đại lượng liên tục.

4.4. Artifact end-to-end

Artifact canonical `models/house_price_pipeline.joblib` chứa cả feature engineering, preprocessing và estimator. Vì vậy app/backend không cần gọi thủ công `engineer_features()` trước khi predict. Thiết kế này giảm nguy cơ training-serving skew, tức khác biệt giữa cách xử lý dữ liệu lúc train và lúc deploy.



Hình: Kiến trúc artifact end-to-end

5. Modeling và Tuning

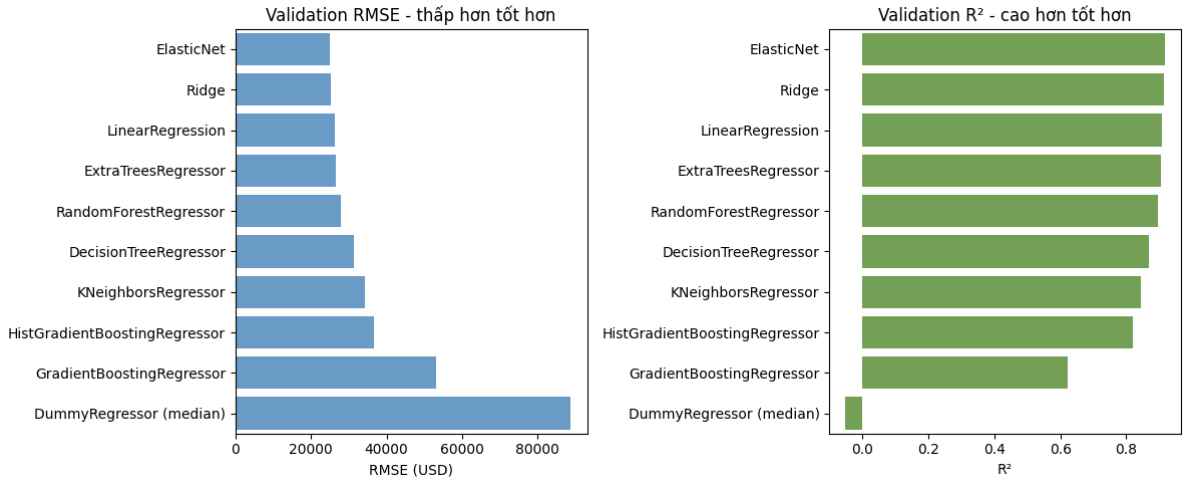
5.1. Nhóm mô hình được benchmark

Benchmark bao gồm DummyRegressor để làm baseline, các mô hình tuyến tính như LinearRegression, Ridge và ElasticNet, nhóm non-linear như KNN, Decision Tree, Random Forest, Extra Trees, Gradient Boosting và HistGradientBoosting. Việc đưa nhiều nhóm mô hình vào benchmark giúp so sánh bias-variance và kiểm tra xem mô hình tuyến tính có đủ mạnh sau feature engineering hay không.

Bảng: Validation benchmark các mô hình

Model	Train_RMSE	RMSE	MAE	R2	RMSLE	RMSE_gap	Relative_to_dummy_RMSE_improvement_pct
ElasticNet	20,829.18	24,855.66	15,376.31	0.9172	0.1202	4,026.48	71.95
Ridge	22,597.60	25,213.20	15,384.61	0.9148	0.1212	2,615.60	71.55
LinearRegression	16,531.38	26,276.88	16,712.36	0.9075	0.1542	9,745.50	70.35
ExtraTreesRegressor	0.0000	26,552.11	16,503.17	0.9056	0.1404	26,552.11	70.04
RandomForestRegressor	12,994.74	27,861.07	14,376.89	0.8960	0.1200	14,866.33	68.56

Model	Train_RMSE	RMSE	MAE	R2	RMSLE	RMSE_gap	Relative_to_dummy_RMSE_improvement_pct
DecisionTreeRegressor	21,878.49	31,457.93	22,219.56	0.8674	0.1707	9,579.45	64.50
KNeighborsRegressor	0.0032	34,184.20	19,291.92	0.8435	0.1537	34,184.20	61.42
HistGradientBoostingRegressor	31,297.97	36,644.32	18,140.17	0.8201	0.1415	5,346.35	58.65
GradientBoostingRegressor	46,569.14	53,014.24	27,105.12	0.6235	0.2117	6,445.10	40.17
DummyRegressor (median)	81,264.53	88,612.20	58,271.19	-0.0519	0.4178	7,347.67	0.0000



Hình: So sánh validation RMSE của các mô hình

5.2. Tuning protocol

Tuning chỉ dùng train/validation theo protocol đã định. Search space và số candidate được giới hạn để project tái lập nhanh trong môi trường học thuật. Test không tham gia tuning. Bảng tuning lưu validation metrics, CV mean/std, thời gian search và tham số tốt nhất.

Bảng: Kết quả tuning

Model	RMSE	MAE	R2	RMSLE	CV_RMSE_mean	CV_RMSE_std	CV_train_RMSE_mean	Search_seconds	Candidates_evaluated	Best_params
Elastic Net (Tuned)	24,855.66	15,376.31	0.9172	0.1202	39,728.85	15,890.45	19,047.56	2.1259	3	{"regressor__model__l1_ratio": 0.2, "regres

Model	RMSE	MAE	R2	RMSLE	CV_RMSE_mean	CV_RMSE_std	CV_train_RMSE_mean	Search_seconds	Candidates_evaluated	Best_params
										sor__model__alpha": 0.001}
Ridge (Tuned)	25,362.19	15,209.59	0.9138	0.1191	41,703.92	19,210.70	24,903.71	1.6158	3	{"regressor__model__alpha": 28.48035868435799}

5.3. Executable model selection policy

Model cuối là ElasticNet với tham số `{'regressor__model__l1_ratio': 0.2, 'regressor__model__alpha': 0.001}`. Quyết định model được khóa bằng logic trong `src/house_price/selection.py` và `outputs/model_selection_policy.json`, không chỉ là diễn giải trong báo cáo. Với chính sách hiện tại, tiêu chí executable là validation RMSE; Ridge chỉ được ưu tiên khi nằm trong tolerance 2% so với model tốt nhất và CV RMSE mean không tệ hơn. Các yếu tố triển khai chỉ là cân nhắc định tính sau selection, không được mô tả như tiêu chí code đang thực thi. Test chưa được mở ở thời điểm chọn model.

5.4. CV stability

Bảng: Cross-validation stability của model cuối

CV_RMSE_mean	CV_RMSE_std	CV_MAE_mean	CV_MAE_std	CV_R2_mean	CV_R2_std	CV_train_RMSE_mean
39,728.85	15,890.45	16,070.74	1,377.37	0.7237	0.1820	19,047.56

CV RMSE mean hiện là 39,728.85 USD với std 15,890.45 USD. Std không nên bị che giấu vì nó cho biết độ nhạy của model với split trong training. Trong dữ liệu nhà ở, một số căn giá cao có thể tạo high-leverage observations, khiến fold chứa nhiều căn như vậy khó dự báo hơn. Vì vậy báo cáo diễn giải model là lựa chọn hợp lý theo trade-off, không phải model hoàn hảo.

6. Final Evaluation

6.1. Test unlock sau model lock

Sau khi model và hyperparameter đã được khóa, tập test mới được mở. Kết quả test sau đó chỉ dùng để báo cáo final performance, residual analysis, error analysis và explainability. Nếu xem test rồi quay lại đổi model, toàn bộ test metric sẽ mất giá trị khách quan.

6.2. Metric cuối

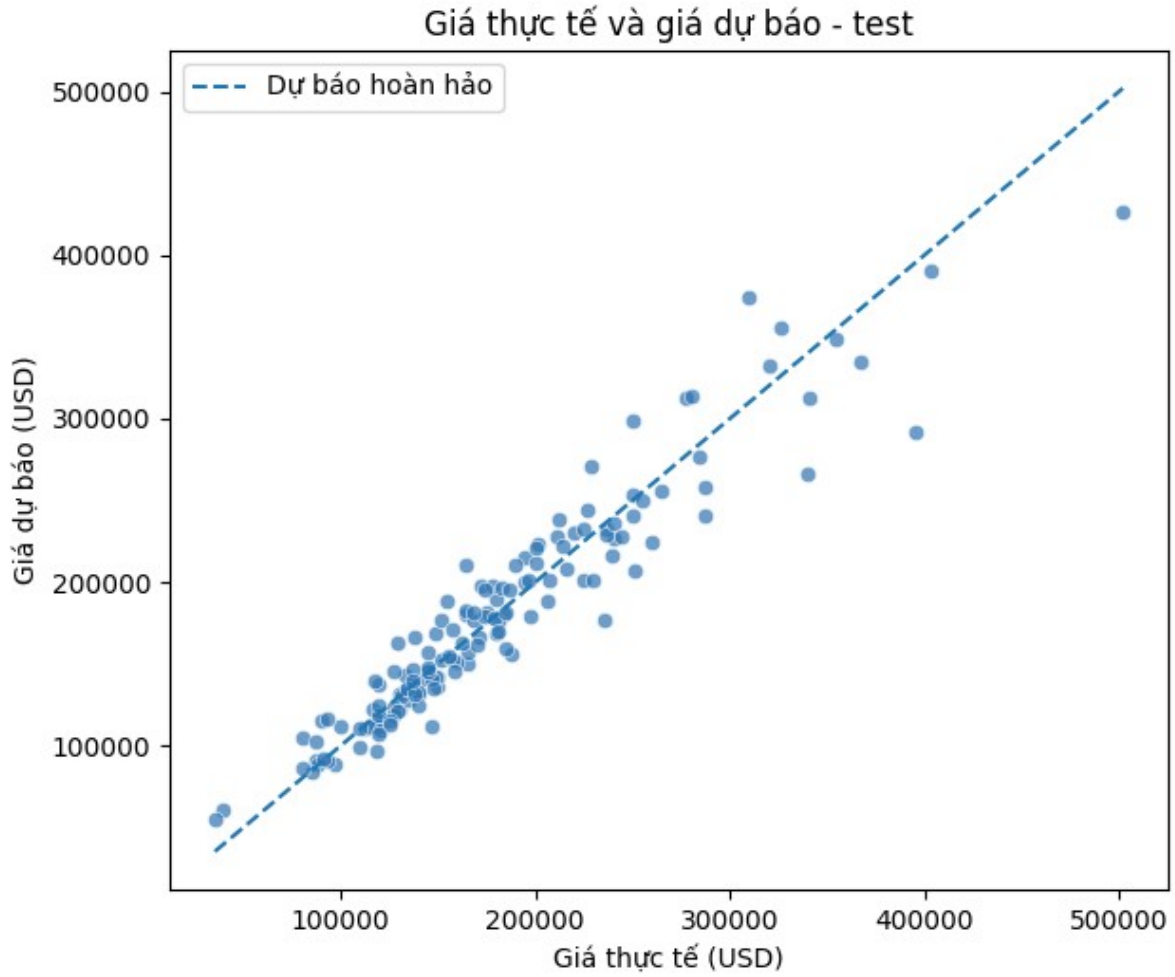
Bảng: Metric validation và test từ outputs/summary.json

Metric	Validation	Test
RMSE	24,855.66 USD	22,228.60 USD
MAE	15,376.31 USD	15,452.34 USD
R2	0.9172	0.9076
RMSLE	0.1202	0.1177

Test RMSE đạt 22,228.60 USD; MAE đạt 15,452.34 USD; R2 đạt 0.9076; RMSLE đạt 0.1177. RMSE nhạy với lỗi lớn nên thể hiện rủi ro ở các căn dự báo sai mạnh. MAE dễ giải thích hơn vì là sai số tuyệt đối trung bình theo USD. RMSLE phù hợp khi target lệch phải và sai số tương đối quan trọng.

6.3. Confidence interval

Bootstrap 95% CI cho RMSE nằm trong khoảng 18,167.24 USD đến 26,757.93 USD; CI cho MAE nằm trong khoảng 12,925.41 USD đến 18,188.23 USD. Khoảng tin cậy giúp tránh kết luận quá chắc chắn từ một test split duy nhất.



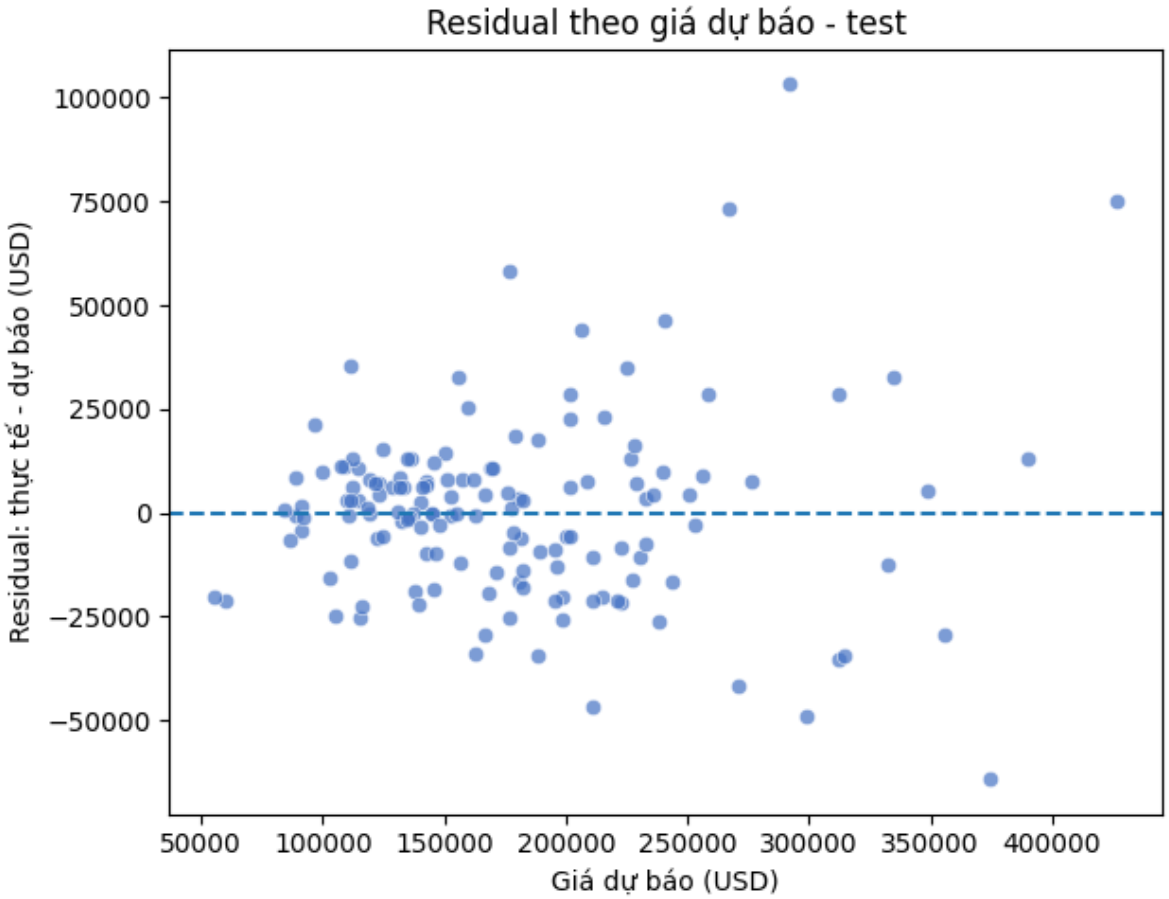
Hình: Actual versus predicted trên test

7. Diagnostic và Explainability

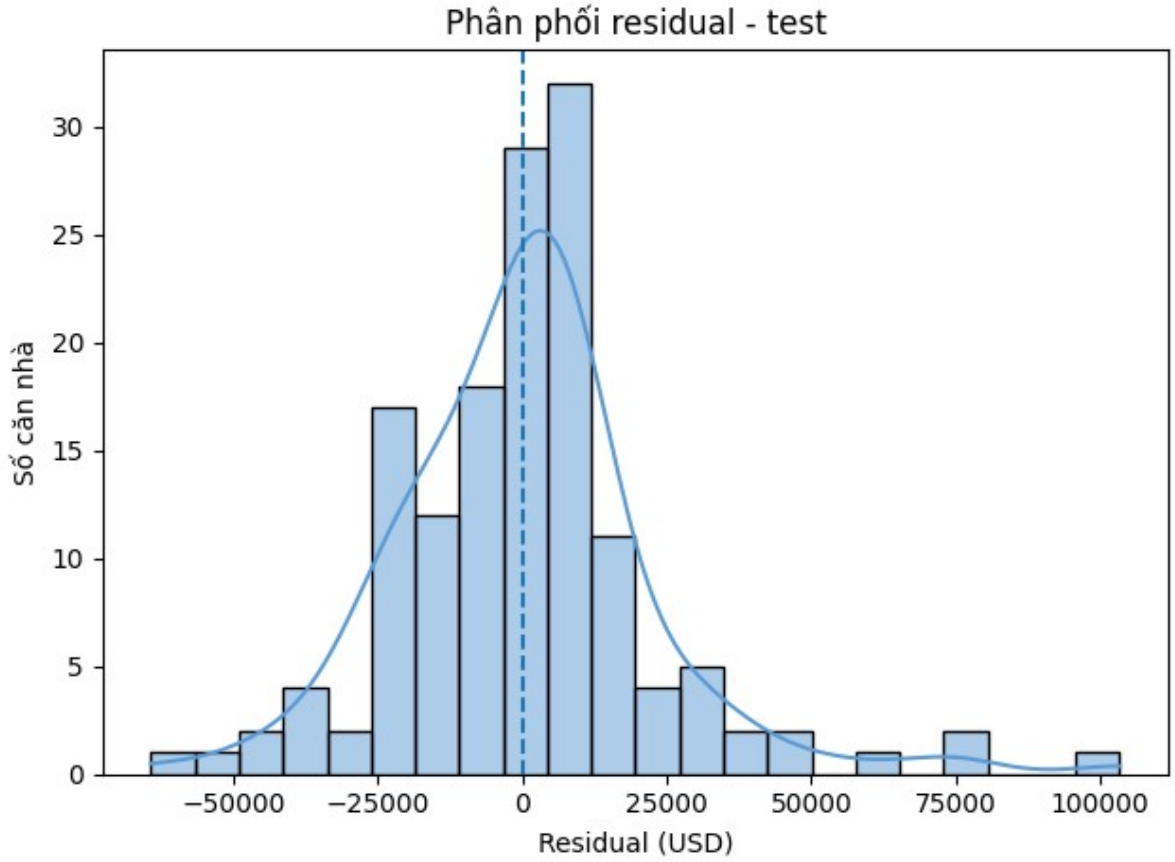
7.1. Residual analysis

Residual analysis cho biết mô hình sai theo hướng nào. Nếu residual tăng theo predicted value, mô hình có thể có bias theo phân khúc giá. Nếu residual phân phối lệch mạnh hoặc có đuôi dài, cần thảo luận outlier

và high-price houses. Đây là lý do báo cáo không chỉ đưa bốn metric cuối mà còn có hình residual và bảng sai số.



Hình: Residual versus predicted value

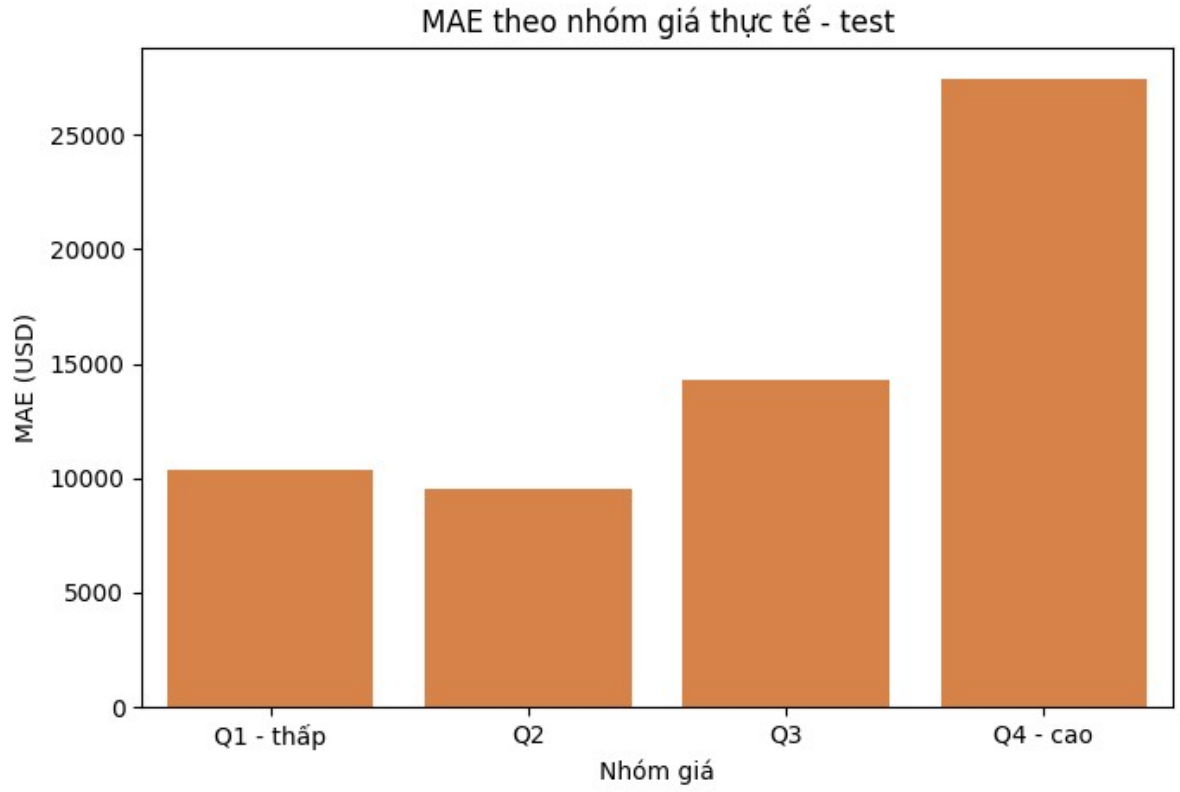


Hình: Phân phối residual

7.2. Error by price quartile

Bảng: Sai số theo price quartile

PriceQuartile	Count	MAE	Median_AE	MAPE
Q1 - thấp	38	10,360.39	7,556.78	11.52
Q2	35	9,551.12	6,455.40	6.4911
Q3	36	14,265.90	11,862.03	7.7748
Q4 - cao	37	27,418.52	22,806.62	9.4601



Hình: MAE theo nhóm giá

Phân tích theo quartile giúp phát hiện nhóm giá nào khó dự báo. Trong nhiều bộ dữ liệu nhà ở, phân khúc giá cao thường có ít mẫu hơn và chịu ảnh hưởng bởi đặc điểm đặc biệt, khiến sai số tuyệt đối lớn hơn. Nếu chỉ báo cáo metric trung bình, hạn chế này dễ bị che khuất.

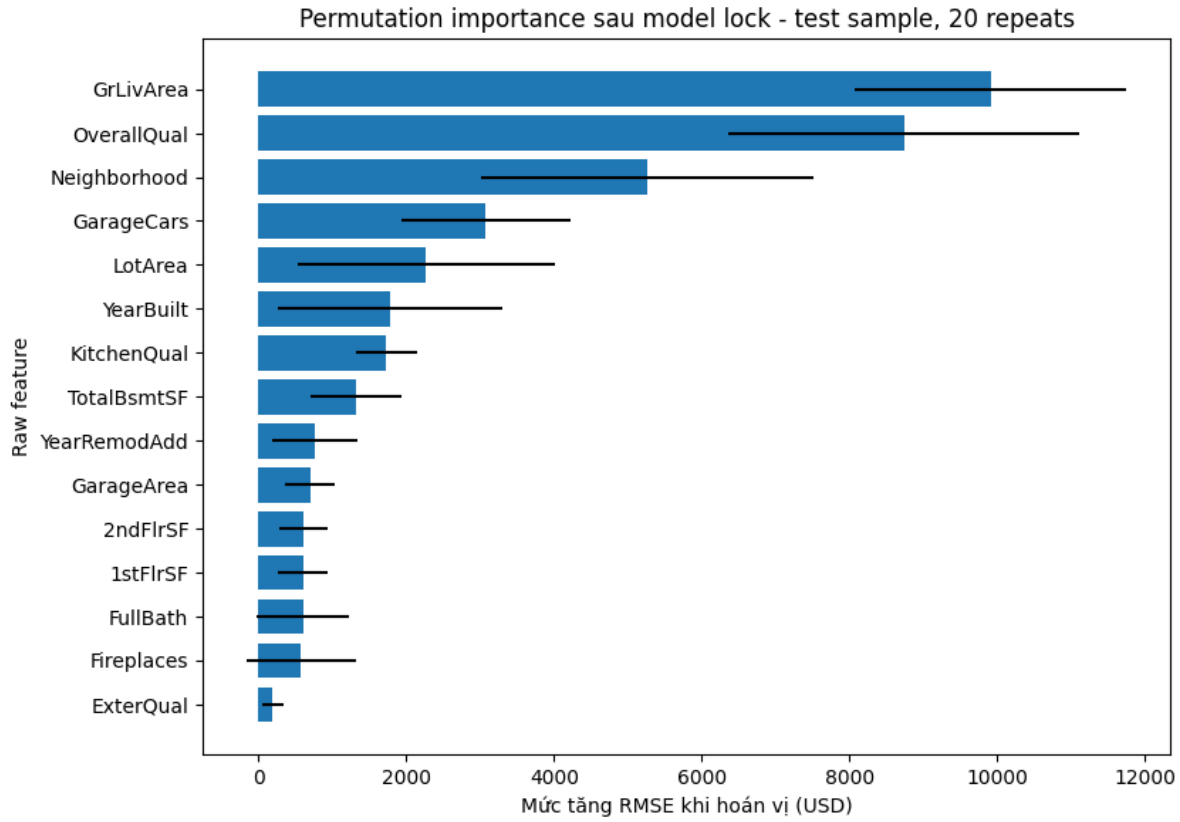
7.3. Largest errors

Bảng: Top 10 absolute errors

Id	ActualSalePrice	PredictedSalePrice	Residual	AbsoluteError	AbsolutePercentageError	PriceQuartile	Neighborhood	OverallQual	GrLivArea	YearBuilt
775	395000	291,666.04	103,333.96	103,333.96	26.16	Q4 - cao	NridgHt	8	1973	2006
179	501837	426,699.77	75,137.23	75,137.23	14.97	Q4 - cao	StoneBr	9	2234	2008
378	340000	266,687.31	73,312.69	73,312.69	21.56	Q4 - cao	Somerst	8	2466	2004
1443	310000	374,252.33	-64,252.33	64,252.33	20.73	Q4 - cao	Somerst	10	2007	2008
1169	235000	176,767.82	58,232.18	58,232.18	24.78	Q4 - cao	Edwards	6	2108	1935
5	250000	299,110.23	-49,110.23	49,110.23	19.64	Q4 - cao	NoRidge	8	2198	2000
1212	164000	210,670.87	-46,670.87	46,670.87	28.46	Q3	Gilbert	8	1752	1988
1038	287000	240,498.37	46,501.63	46,501.63	16.20	Q4 - cao	CollgCr	8	2263	2001
865	250580	206,489.41	44,090.59	44,090.59	17.60	Q4 - cao	Somerst	7	1372	2007

Id	ActualSalePrice	PredictedSalePrice	Residual	AbsoluteError	AbsolutePercentageError	PriceQuartile	Neighborhood	OverallQual	GrLivArea	YearBuilt
336	228950	270,718.81	-41,768.81	41,768.81	18.24	Q4 - cao	Timber	5	1786	1965

7.4. Permutation importance



Hình: Permutation importance với 20 repeats

Bảng: Top permutation importance

Feature	Importance_mean	Importance_std	Repeats	Sample_size
GrLivArea	9,910.12	1,840.31	20	50
OverallQual	8,734.33	2,375.29	20	50
Neighborhood	5,256.71	2,250.60	20	50
GarageCars	3,076.45	1,148.80	20	50
LotArea	2,269.53	1,749.56	20	50
YearBuilt	1,777.64	1,522.11	20	50
KitchenQual	1,730.04	419.12	20	50
TotalBsmtSF	1,321.19	620.20	20	50
YearRemodAdd	762.10	571.38	20	50
GarageArea	699.45	338.41	20	50
2ndFlrSF	607.39	329.23	20	50
1stFlrSF	604.13	338.36	20	50

Permutation importance được tính sau model lock, với nhiều repeats để có độ lệch chuẩn thật. Kết quả đo mức tăng RMSE khi một raw feature bị hoán vị, do đó phản ánh mức độ mô hình phụ thuộc vào feature trong dữ liệu hiện tại. Đây không phải causal importance. Khi các feature tương quan, importance có thể bị chia sẻ hoặc giảm.

8. Hệ thống phần mềm

8.1. Core architecture sau refactor

Kiến trúc mới tách rõ responsibilities. `scripts/train_full_project.py` chỉ còn vai trò orchestration: load data, gọi EDA, benchmark, tuning, selection, final evaluation và lưu artifact. Các logic tái sử dụng nằm trong `src/house_price`: `eda.py`, `tuning.py`, `selection.py`, `diagnostics.py`, `explainability.py`, `reporting.py`, cùng các module cũ như `features.py`, `preprocessing.py` và `artifacts.py`.

8.2. Artifact boundary

Artifact SHA-256 là `5101114637e8001c91f06a42737cd3cc8ee143cb1d6a3a27631a41b48f3efcf1`. Backend và Streamlit chỉ load artifact này, không fit lại model. Prediction flow là raw input -> schema validation -> artifact predict -> USD output. Thiết kế này phù hợp với demo lớp học vì dễ giải thích, dễ kiểm thử và giảm rủi ro deploy lệch train.

8.3. Backend và Streamlit

- `backend/service.py` chứa `predict_one` và `predict_batch`.
- Batch prediction giữ extra columns trong output nhưng không đưa extra columns vào model.
- App Streamlit có Quick Input, Full Input, Batch CSV và Model Information.
- Streamlit phải được launch thật ở máy có package trước khi claim READY TO SUBMIT.

9. Testing và Reproducibility

9.1. Test suite

Sau refactor, test suite chạy 67 tests passed với source coverage 99.9% trên phạm vi src/house_price + backend. Các test mới kiểm tra training script chỉ còn orchestration, selection policy, permutation importance repeat count, diagnostic outputs và reporting helpers. Điều này trực tiếp xử lý lỗi kiến trúc còn lại của bản ULTIMATE trước.

- Data tests: schema, target, ID uniqueness, split overlap, semantic missing.
- Feature tests: deterministic outputs, expected features, no target leakage.
- Artifact tests: raw input direct prediction, serialization, missing and unknown category.
- Backend tests: single prediction, batch prediction, missing required column, invalid data.
- Architecture tests: no large scientific functions in training script.
- Consistency tests: model, metrics and documentation alignment.

9.2. Coverage interpretation

Coverage cao không chứng minh không có bug, nhưng coverage thấp ở module quan trọng là dấu hiệu rủi ro. Vì vậy mục tiêu của bản này không phải thêm test vô nghĩa, mà là test các vùng từng yếu: diagnostics, EDA, explainability, reporting, selection policy và architecture. Một số dòng ngoại lệ hoặc branch hiếm vẫn có thể chưa được chạm, nhưng tổng thể coverage đã vượt ngưỡng 95%.

9.3. Reproducibility workflow

Một clean checkout có thể chạy theo thứ tự: cài môi trường Python >=3.11, chạy data audit, train pipeline, build metadata, execute notebooks, run tests, build report, build slides và smoke test backend. Nếu

Streamlit package có sẵn, chạy `streamlit run app/Home.py` để verify UI. Các script build report/slide hiện tạo artifact thật, không chỉ kiểm tra file tồn tại.

10. Triển khai và Demo

10.1. Local demo flow

Demo nên đi từ bài toán đến sản phẩm. Đầu tiên giới thiệu dataset và metric, sau đó nhấn mạnh leakage protocol. Tiếp theo mở model selection policy và summary.json để chứng minh model đã khóa trước test. Cuối cùng chạy Streamlit hoặc backend smoke test để dự báo một căn nhà và batch CSV.

10.2. Deployment notes

- Local command: `streamlit run app/Home.py`.
- Cloud entrypoint: `app/Home.py`.
- Không cần secrets.
- Không deploy khi thiếu model/schema/summary.
- Luôn kiểm tra log để chắc app không train lại model.

11. Hạn chế và hướng phát triển

11.1. Hạn chế

- Dữ liệu thuộc Ames/Iowa, không đại diện trực tiếp cho thị trường Việt Nam.
- High-price houses có thể gây sai số lớn và CV variance.
- Split gốc được cung cấp sẵn, không có script tạo split ban đầu.
- Streamlit cần verify ở môi trường có package trước khi nộp chính thức.
- Model không thay thế thẩm định chuyên nghiệp.

11.2. Hướng phát triển

Các hướng phát triển hợp lý gồm deploy Streamlit thật, bổ sung UI testing, cải thiện monitoring drift, thu thập dữ liệu mới, kiểm thử với external validation set và bổ sung giải thích bằng các phương pháp chuyên sâu hơn. Không nên ghi hướng phát triển trùng với kỹ thuật đã dùng như “thử log target” vì log target hiện đã nằm trong artifact.

12. Kết luận

Project cuối giữ được metric ổn định của ElasticNet, đồng thời sửa điểm yếu kiến trúc bằng cách tách logic khỏi training script. Test suite đạt 67 tests passed và coverage 99.9%; báo cáo, slide và README được rebuild từ canonical outputs. Về mặt kỹ thuật, project đã tiến gần chuẩn 9.6-10 khi chấm gắt. Điểm chưa nên tự bịa là thông tin hành chính và trạng thái Streamlit nếu chưa launch trên máy có package.

13. Phân tích chuyên sâu bổ sung

13.1. Vì sao không chỉ tối ưu test RMSE

Trong một đồ án Machine Learning, cách dễ làm metric đẹp là thử nhiều mô hình, quan sát test rồi chọn mô hình có test RMSE thấp nhất. Cách làm đó sai về phương pháp vì test không còn độc lập. Bản hiện tại cố tình không thay đổi split, không tune trên test và không đổi model chỉ để giảm sai số test. Giá trị của project nằm ở experimental protocol có thể bảo vệ: train dùng để fit, validation dùng để chọn, test dùng để

đánh giá cuối. Khi giảng viên hỏi vì sao không chọn một mô hình khác nếu test thấp hơn, câu trả lời đúng là test không được phép là công cụ ra quyết định.

Điều này cũng giải thích vì sao báo cáo trình bày cả validation, CV, test và error analysis thay vì một con số duy nhất. Tuy nhiên, cần phân biệt rõ giữa tiêu chí selection executable và phân tích hậu nghiệm: code chọn model theo validation RMSE cùng Ridge tolerance/CV tie-break đã công bố; test và error analysis chỉ dùng để đánh giá sau model lock.

13.2. Vai trò của refactor kiến trúc

Trước refactor, training script dài hơn bốn trăm dòng và chứa nhiều hàm khoa học như EDA, tuning, final evaluation và permutation importance. Về mặt chạy code, điều này vẫn có thể tạo kết quả đúng. Tuy nhiên, về kiến trúc phần mềm, nó làm giảm khả năng tái sử dụng và khó chứng minh `src/house_price` là single source of truth. Sau refactor, các trách nhiệm chính được đưa về module riêng: `eda.py` tạo artifact EDA; `tuning.py` benchmark và tune; `selection.py` chứa policy chọn model; `diagnostics.py` xử lý final holdout; `explainability.py` tính permutation importance; `reporting.py` lưu bảng, hình và summary. Training script chỉ điều phối các bước.

Thiết kế này giúp notebook, test và tài liệu có thể gọi chung cùng logic. Nếu một quy tắc selection thay đổi, người phát triển sửa trong `selection.py` và test policy tương ứng, thay vì sửa rải rác trong notebook, script và báo cáo. Đây là tiêu chí quan trọng khi chấm gắt vì project không chỉ là một notebook chạy được mà là một repository có kiến trúc.

13.3. Phân tích từng nhóm mô hình

DummyRegressor là baseline tối thiểu; nếu mô hình học máy không vượt baseline thì toàn bộ pipeline không có giá trị dự báo. LinearRegression cung cấp mốc tuyến tính không regularization, dễ diễn giải nhưng nhạy với multicollinearity sau one-hot encoding. Ridge thêm L2 regularization, thường ổn định hơn khi có nhiều biến tương quan. ElasticNet kết hợp L1 và L2, có thể vừa regularize vừa giảm ảnh hưởng của một số hệ số không cần thiết, phù hợp với dữ liệu nhiều feature sau encoding.

KNN là mô hình dựa trên khoảng cách, có thể bắt pattern phi tuyến cục bộ nhưng dễ bị ảnh hưởng bởi scale và sparse one-hot dimensions. Decision Tree có khả năng mô tả tương tác nhưng dễ overfit. Random Forest và Extra Trees giảm variance bằng ensemble, nhưng có thể nặng hơn và khó giải thích tuyến tính. Gradient Boosting và HistGradientBoosting có khả năng mạnh trên tabular data, nhưng cần tuning cẩn thận và có thể overfit hoặc làm deployment phức tạp hơn. Bảng benchmark giúp người chấm thấy nhóm đã cân nhắc nhiều lựa chọn, không chọn model một cách tùy tiện.

13.4. Vì sao cần kiểm thử selection policy

Một lỗi thường gặp là tài liệu nói model được chọn vì nhiều tiêu chí, nhưng code chỉ chọn model có validation RMSE thấp nhất. Bản hiện tại bổ sung test cho `select_final_model` để mô phỏng các trường hợp synthetic: Ridge chỉ được chọn khi trong ngưỡng validation tolerance và CV không tệ hơn; nếu CV của Ridge xấu hơn, policy giữ model tốt hơn. Test này không dùng dữ liệu thật để làm metric đẹp, mà kiểm tra logic ra quyết định có khớp với policy đã công bố hay không.

Nhờ đó, khi báo cáo nói selection policy là executable, đây không chỉ là câu văn. Người chấm có thể mở `src/house_price/selection.py` và `tests/test_architecture_refactor.py` để thấy quy tắc được biểu diễn bằng code và được kiểm thử. Đây là mức minh bạch cao hơn so với việc ghi lý do chọn model trong README rồi không có bằng chứng.

13.5. Ý nghĩa của consistency checker

Trong các bản trước từng có lỗi nghiêm trọng: README ghi một model, model card ghi model khác; audit ghi test count cũ; Word và slide có metric không đồng nhất. Consistency checker được dùng để giảm rủi ro này. Checker cần đọc nhiều nguồn: JSON canonical, Markdown docs, DOCX text, PPTX text và

audit logs. Mục tiêu không phải thay thế review thủ công, mà là phát hiện sớm các contradiction dễ bỏ sót khi chỉnh nhiều artifact.

Một consistency checker tốt phải kiểm tra model name, hyperparameters, validation metrics, test metrics, artifact hash, test count và coverage. Nó cũng cần phân biệt nhắc đến Ridge như candidate model hợp lệ với claim sai rằng Ridge là mô hình cuối. Nhờ đó, tài liệu có thể vừa đầy đủ vừa không mâu thuẫn.

13.6. Kiểm thử không chỉ là coverage

Source coverage 99.9% trên phạm vi src/house_price + backend là một tín hiệu tốt nhưng không phải bảo đảm tuyệt đối. Điều quan trọng hơn là test có chạm đúng rủi ro hay không. Bản hiện tại test dữ liệu, feature engineering, artifact raw input, backend batch, unknown category, missing value, model selection, permutation importance, diagnostic outputs và kiến trúc training script. Đây là các điểm từng bị phản biện trong audit, nên test mới có giá trị thực tế.

Một số branch như lỗi file thiếu hoặc ngoại lệ hiếm có thể chưa cần chạy trong từng test nếu đã có cơ chế báo lỗi thân thiện. Tuy nhiên, các chức năng cốt lõi như artifact predict raw input, policy selection và no duplicate joblib artifact phải có test trực tiếp. Khi bảo vệ, nhóm nên nói rõ coverage cao không thay thế review, nhưng nó là evidence cho quá trình kỹ thuật nghiêm túc.

13.7. Diễn giải hạn chế của Streamlit

Streamlit là lớp trình diễn sản phẩm, nhưng trạng thái PASS chỉ hợp lệ khi ứng dụng được launch thật. Nếu môi trường container không có executable hoặc package, báo cáo không được ghi app PASS tuyệt đối. Thay vào đó, project có thể chứng minh backend smoke test, artifact loading và prediction service hoạt động; còn UI launch cần được thực hiện trên máy cá nhân hoặc môi trường cloud có cài Streamlit. Đây là cách trung thực và an toàn hơn so với ghi một claim không có log.

Khi nhóm chuẩn bị nộp, cần chạy `streamlit run app/Home.py`, kiểm tra log không traceback, thử quick prediction, full input, batch CSV và model information. Nếu có screenshot hoặc deployment URL thật, bổ sung vào README/slide. Nếu chưa có, giữ trạng thái technical backend ready nhưng administrative/demo verification pending.

13.8. Ý nghĩa của artifact end-to-end

Artifact end-to-end là yêu cầu quan trọng vì trong nhiều đồ án sinh viên, notebook train một pipeline nhưng app lại tự viết lại feature engineering. Khi đó chỉ cần một công thức khác hoặc một cột thiếu là prediction ở app không còn giống training. Artifact hiện tại nhận raw 79 features và tự thực hiện AmesFeatureEngineer, ColumnTransformer, estimator và inverse log target. Test trực tiếp gọi `model.predict(raw_dataframe)` mà không chạy `engineer_features()` bên ngoài.

Thiết kế này giúp giảm training-serving skew, làm deploy đơn giản và tăng tính bảo vệ. Khi giảng viên hỏi app có dùng cùng pipeline với training không, nhóm có thể chỉ ra artifact joblib, metadata `contains_feature_engineering=True`, test artifact direct predict và backend service chỉ load model chứ không fit lại.

13.9. Chiến lược đọc kết quả khi bảo vệ

Khi trình bày trước giảng viên, nhóm nên tránh đọc lần lượt tất cả bảng mà cần kể một câu chuyện thống nhất. Câu chuyện nên bắt đầu từ bài toán định giá nhà, chuyển sang lý do dữ liệu cần xử lý cẩn thận, sau đó nhấn mạnh nguyên tắc chống leakage. Tiếp theo, nhóm trình bày vì sao nhiều mô hình được benchmark, vì sao model cuối được chọn theo policy và vì sao test chỉ xuất hiện ở cuối. Cách trình bày này cho thấy nhóm hiểu phương pháp, không chỉ chạy code.

Một chiến lược trả lời tốt là luôn trỏ về evidence. Nếu bị hỏi metric, mở `outputs/summary.json`. Nếu bị hỏi feature, mở `outputs/tables/feature_engineering.csv`. Nếu bị hỏi model selection, mở

`outputs/model_selection_policy.json` và `src/house_price/selection.py`. Nếu bị hỏi deployment, mở `models/input_schema.json` và backend service. Việc trả lời dựa trên evidence giúp tăng độ tin cậy và tránh tranh luận cảm tính.

13.10. Phân biệt lỗi kỹ thuật và lỗi hành chính

Dự án hiện có thể đạt trạng thái technical ready nếu code, test, notebook, artifact, report và slide đã đồng bộ. Tuy nhiên, submission readiness còn phụ thuộc các thông tin hành chính như tên nhóm, giảng viên, lớp học phần, thành viên và mã sinh viên. Prompt yêu cầu không tự bịa các thông tin này, nên báo cáo giữ placeholder rõ ràng. Điều này không phải lỗi kỹ thuật, nhưng là blocker trước khi nộp chính thức.

Do đó, audit cuối phải tách ba trạng thái: technical readiness, administrative information required và fully ready to submit. Nếu thông tin hành chính còn thiếu, không được tự chấm submission readiness 9.6 trở lên. Đây là cách trung thực và phù hợp với yêu cầu ban đầu.

13.11. Kiểm soát stale evidence

Một rủi ro lớn của project nhiều lần chỉnh sửa là audit log bị cũ. Nếu giữ nhiều thế hệ log với test count hoặc coverage khác nhau, người chấm có thể nghi ngờ toàn bộ evidence. Vì vậy sau refactor, các file audit mới phải được tạo lại và `FINAL_AUDIT_REPORT.md` phải ghi actual run gần nhất. Các log cũ nếu giữ trong repository cần đánh dấu hoặc thay thế; tốt nhất chỉ giữ log final cần thiết.

Nguyên tắc là không dùng audit report làm nguồn sự thật nếu chưa đối chiếu lại với command hiện tại. Mọi claim PASS phải gắn với command chạy thật, thời điểm chạy, kết quả và file evidence. Đây cũng là lý do final audit phải giải nén chính ZIP cuối sang thư mục mới và kiểm tra lại như bên thứ ba.

13.12. Cách đọc các bảng output

`validation_model_comparison.csv` cho biết hiệu năng các model trước tuning hoặc theo cấu hình mặc định có kiểm soát. `tuning_results.csv` ghi kết quả tìm kiếm hyperparameter cho các ứng viên chính. `selected_model_cv_stability.csv` tách riêng độ ổn định CV của model cuối. `test_metrics.csv` là kết quả holdout sau model lock. `test_predictions.csv`, `error_by_price_quartile.csv`, `error_by_neighborhood.csv` và `top10_absolute_errors.csv` phục vụ phần diagnostic.

Khi các bảng này được đọc cùng nhau, người xem có thể tái hiện toàn bộ quy trình: model nào được thử, model nào được tune, model nào được chọn, test ra sao và model sai mạnh ở đâu. Đây là sự khác biệt giữa một bài notebook chỉ in metric và một project có audit trail.

13.13. Tại sao không nên che giấu sai số lớn

Các căn có absolute error lớn nhất thường là phần dễ bị hỏi trong bảo vệ. Che giấu chúng sẽ làm báo cáo thiếu trung thực. Thay vào đó, nhóm nên phân tích chúng như limitation: nhà giá cao, neighborhood đặc biệt, diện tích lớn hoặc chất lượng vượt trội có thể nằm ở vùng ít mẫu. Model dự báo dựa trên pattern học được từ training; khi một căn có cấu hình hiếm, sai số tăng là điều hợp lý.

Phân tích top errors giúp đề xuất hướng cải thiện: thu thập thêm dữ liệu phân khúc cao, thêm đặc trưng vị trí chi tiết hơn, thử external validation hoặc mô hình chuyên biệt theo nhóm giá. Điều này chứng minh nhóm hiểu sai số và biết cách phát triển tiếp, thay vì chỉ tối ưu con số trung bình.

Một điểm cuối cùng cần nhấn mạnh là báo cáo không đánh đồng độ dài với chất lượng. Các phần bổ sung được viết để trả lời các câu hỏi phản biện thực tế: dữ liệu đến từ đâu, model được chọn thế nào, test được mở khi nào, sai số lớn nằm ở đâu, artifact deploy ra sao và audit evidence có đáng tin hay không. Nhờ đó, phần trình bày không chỉ nhiều trang hơn mà còn có chiều sâu bảo vệ tốt hơn.

Tài liệu tham khảo

- De Cock, D. (2011). Ames, Iowa: Alternative to the Boston Housing Data as an End of Semester Regression Project. Journal of Statistics Education.
- Kaggle. House Prices - Advanced Regression Techniques dataset.
- Pedregosa et al. (2011). Scikit-learn: Machine Learning in Python.
- scikit-learn documentation: Pipeline, ColumnTransformer, TransformedTargetRegressor, ElasticNet.

Phụ lục A. Thông tin cần điền

- Tên nhóm: [CẦN CUNG CẤP]
- Giảng viên: [CẦN CUNG CẤP]
- Lớp học phần: [CẦN CUNG CẤP]
- Thành viên + mã sinh viên: [CẦN CUNG CẤP]
- Tỷ lệ đóng góp: [CẦN CUNG CẤP]

Phụ lục B. Mapping file quan trọng

Bảng: File quan trọng sau refactor

File	Vai trò
README.md	Hướng dẫn tổng quan, workflow, metric, deployment và troubleshooting
outputs/summary.json	Nguồn số liệu chuẩn duy nhất
outputs/model_selection_policy.json	Policy chọn model đã khóa trước test
models/house_price_pipeline.joblib	Canonical artifact end-to-end
src/house_price/selection.py	Executable selection policy
src/house_price/tuning.py	Benchmark/tuning reusable logic
src/house_price/diagnostics.py	Final diagnostics after model lock
scripts/train_full_project.py	Orchestrator only

Phụ lục C. Lệnh tái lập

```
python scripts/audit_data.py

python scripts/train_full_project.py

python scripts/build_metadata.py

python scripts/execute_notebooks.py

coverage run --source=src/house_price,backend -m pytest -q &&
coverage report -m

python scripts/check_project_consistency.py

python scripts/build_report.py

python scripts/build_slides.py
```

```
streamlit run app/Home.py
```

Phụ lục D. Checklist bảo vệ

- Mở outputs/summary.json để xác nhận model cuối và metric.
- Mở outputs/model_selection_policy.json để chứng minh test chưa dùng trong selection.
- Mở src/house_price/selection.py để chỉ ra policy executable.
- Mở tests/test_architecture_refactor.py để chứng minh training script orchestration.
- Mở coverage report để chứng minh coverage >=95%.
- Mở DEFENSE_QA.md để luyện câu hỏi phản biện.